



UNIVERSIDAD DE MÁLAGA



Grado en Ingeniería del Software

Aplicación para la gestión integral del proceso de detección
de metano con tecnología TDLAS

Application for the comprehensive management of the
methane detection process with TDLAS technology

Realizado por
Alejandro Román Sánchez

Tutorizado por
Javier González Monroy
Cipriano Galindo Andrades

Departamento
Ingeniería de Sistemas y Automática
UNIVERSIDAD DE MÁLAGA

MÁLAGA, junio de 2025



UNIVERSIDAD
DE MÁLAGA



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA INFORMÁTICA
GRADUADO EN INGENIERÍA DEL SOFTWARE

**Aplicación para la gestión integral del proceso de
detección de metano con tecnología TDLAS**

**Application for the comprehensive management of the
methane detection process with TDLAS technology**

Realizado por
Alejandro Román Sánchez

Tutorizado por
Javier González Monroy
Cipriano Galindo Andrades

Departamento
Ingeniería de Sistemas y Automática

UNIVERSIDAD DE MÁLAGA
MÁLAGA, JUNIO DE 2025

Fecha defensa: 7 de julio de 2025

Abstract

Keywords: TDLAS, Methane, ROS2, Environmental Monitoring, Remote Detection

Resumen

Palabras clave: TDLAS, Metano, ROS₂, Monitorización ambiental,
Detección remota

Índice

1. Introducción	7
1.1. Motivación	7
1.2. Objetivos	8
1.3. Tecnologías usadas	8
1.4. Estructura del documento	9
2. Estado del Arte y Antecedentes	11
2.1. Fundamentos del Metano	11
2.1.1. Características del Metano	11
2.1.2. Aplicaciones Industriales y Ambientales	12
2.2. Introducción a la Tecnología TDLAS	13
2.2.1. Fundamentos físicos de TDLAS	13
2.2.2. Ventajas y Limitaciones de TDLAS	14
2.3. Soluciones existentes para la detección y monitorización de Metano	15
2.3.1. Sistemas de medición estacionarios	15
2.3.2. Sistemas móviles y uso de drones	15
2.3.3. Soluciones de software y automatización existentes	15
2.4. Herramientas y Tecnologías Complementarias	16
2.4.1. ROS (Robot Operating System)	16
2.4.2. Interfaces gráficas y aplicaciones de control	16
2.5. Automatización y control de procesos de aplicaciones ambientales e industriales	17
2.5.1. Conceptos clave de automatización	17
2.5.2. Sistemas actuales de monitorización y control	17
3. Proceso de Desarrollo y Diseño del Sistema	19
3.1. Requerimientos del sistema	19
3.1.1. Requerimientos funcionales	19
3.1.2. Requerimientos no funcionales	22

3.2.	Metodología de Desarrollo	23
3.2.1.	Enfoque de trabajo: Desarrollo Iterativo e Incremental	23
3.2.2.	Herramientas de Gestión Empleadas	24
3.2.3.	Planificación Temporal	25
3.3.	Diseño del Sistema	26
3.3.1.	Arquitectura de Componentes	26
3.3.2.	Arquitectura de Comunicaciones (RSOS2 <->MQTT <->Robots)	30
3.3.3.	Patrón de diseño empleado	33
3.3.4.	Modelo de Datos	35
3.3.5.	Diseño de la interfaz gráfica	37
3.3.6.	Modelos UML del Sistema	40
3.3.7.	Conclusión del Diseño del Sistema	41
4.	Desarrollo e Integración Tecnológica	43
4.1.	Entorno de Desarrollo y Estructura del Proyecto	43
4.2.	Integración de la Interfaz Gráfica con ROS2	47
5.	Conclusions and Futures Lines of Research	49
5.1.	Conclusions	49
5.2.	Future lines of Research	49
6.	Conclusiones y Líneas Futuras	51
6.1.	Conclusiones	51
6.2.	Líneas Futuras	51
Apéndice A. Manual de		
	Instalación	55

Introducción

1.1. Motivación

La creciente preocupación por el cambio climático y sus efectos en nuestro planeta ha hecho que aumenten los esfuerzos por monitorizar y reducir las emisiones de los gases de efecto invernadero, siendo el metano (CH_4) uno de los principales causantes debido a su alto potencial en aumentar la temperatura global [5]. El metano es un compuesto gaseoso inodoro, incoloro y más ligero que el aire, lo que dificulta enormemente su detección, sobre todo en grandes entornos abiertos. Debido a esto, su detección y medición precisa es fundamental para la implementación de medidas eficaces de control y mitigación en sectores ambientales e industriales.

En la actualidad, la tecnología TDLAS (Tunable Diode Laser Absorption Spectroscopy) se presenta como una solución prometedora gracias a su capacidad de detectar concentraciones de gas a distancia y a su alta selectividad, ofreciendo la posibilidad de tomar decisiones rápidas y precisas. Sin embargo, su aplicación práctica en entornos abiertos implica enfrentarse a algunos retos técnicos, como la sincronización entre sensores y reflectores, la delimitación del área de inspección y la gestión efectiva de errores, entre otros [8].

Para satisfacer estas necesidades, es esencial el desarrollo de herramientas digitales que permitan gestionar de manera integral todas las fases del proceso de medición de manera rápida y eficaz. Estas herramientas nos permiten realizar una fácil interpretación y gestión de los datos obtenidos, asegurando una mayor calidad y precisión en las mediciones. Además, la implementación de estas herramientas nos permiten estudiar mejor estos gases y reducir considerablemente los riesgos asociados a la detección de gases en diferentes contextos.

1.2. Objetivos

El objetivo principal de este Trabajo de Fin de Grado es el desarrollo de una aplicación de escritorio que permita la supervisión, control y registro de mediciones derivadas del proceso de detección y medición de metano con sensores TDLAS.

Al ser un producto software, se buscará que la interfaz gráfica sea intuitiva y usable. Esto permitirá que los usuarios, en este caso también llamados operadores, tengan una mejor experiencia. Además, un diseño intuitivo facilita la navegación del usuario y minimiza el riesgo de frustración del mismo.

La aplicación deberá permitir gestionar las distintas fases del proyecto, en las cuales están incluidas el despliegue inicial del sistema, la configuración detallada de experimentos específicos, la toma de los datos obtenidos en la medición a tiempo real y la visualización inmediata de dichos datos. Además, será necesario facilitar el despliegue y sincronización de los dispositivos involucrados, mostrando al usuario el estado de conexión actual o los posibles errores que hayan ocurrido en su defecto.

Por otro lado, la aplicación deberá permitir al usuario delimitar la zona de inspección, siendo deseable que dicha delimitación pueda realizarse mediante un mapa interactivo 2D de vista satelital. Esto proporcionará una definición precisa del área a medir, contribuyendo a mejorar la eficacia y la calidad del proceso de medición.

Finalmente, tendrá que facilitar la recolección, procesamiento y visualización de los datos obtenidos, dando al operador las herramientas necesarias para tomar decisiones informadas a tiempo real, tales como pausar, abortar o reiniciar los procesos, asegurando así la continuidad y eficiencia del sistema.

1.3. Tecnologías usadas

Para llevar a cabo el desarrollo de este proyecto se han empleado diversas tecnologías divididas en varios apartados según su función dentro del trabajo realizado.

En primer lugar, para la creación de la aplicación de escritorio se ha utilizado como lenguaje de programación principal Python. Python es un lenguaje versátil y potente que proporciona una gran flexibilidad y simplicidad, lo que facilita el desarrollo de aplicaciones complejas de manera eficiente. Se ha escogido este lenguaje debido a que ROS (Robot Operating System),

utilizado en este proyecto para la coordinación robótica y que se explicará más adelante, solo puede programarse de forma efectiva en Python o C++. Gracias a su amplia comunidad y sus numerosas librerías, Python permite integrar fácilmente funcionalidades avanzadas necesarias para este tipo de proyectos científicos y técnicos.

En segundo lugar, para el desarrollo de la interfaz gráfica se ha seleccionado la librería PyQt, un conjunto de enlaces de Python para el marco de aplicaciones Qt. PyQt permite desarrollar interfaces gráficas robustas, intuitivas y altamente personalizables, dando una experiencia más eficaz y fluida al usuario. Su versatilidad permite que el usuario pueda interactuar fácilmente con la aplicación, mejorando su experiencia y la usabilidad del sistema.

Por último, dado que el proyecto implica la coordinación y comunicación con sistemas robóticos para la medición del matano, se ha implementado ROS. ROS es un conjunto de herramientas y bibliotecas que proporciona un marco de software especializado para aplicaciones robóticas. Su implementación ha permitido gestionar de manera eficaz y en tiempo real la comunicación con los distintos dispositivos involucrados en la medición, facilitando la suscripción y publicación a tópicos para enviar o recibir mensajes que son críticos para el proceso. Gracias a esto, se ha garantizado una coordinación precisa y efectiva, optimizando además el rendimiento global del sistema.

Además, se han empleado herramientas auxiliares como Balsamiq Wireframes para la creación y validación rápida de mockups visuales de la interfaz gráfica, facilitando así el posterior desarrollo de la aplicación. También se ha empleado Visual Studio Code como editor principal de código, aprovechando sus extensiones especializadas para mejorar la productividad y asegurar altos estándares en la calidad del código.

1.4. Estructura del documento

El siguiente trabajo se estructura en los siguientes capítulos:

1. Este capítulo cuenta con una breve introducción sobre el trabajo realizado donde podemos encontrar la motivación, los objetivos y las tecnologías que se van a usar.
2. Este capítulo presenta una revisión detallada de la tecnología TDLAS, abordando sus fundamentos, características y usos principales. Además, se descri-

ben soluciones existentes relacionadas con la detección y medición del metano, centrándose especialmente en sistemas o aplicaciones similares en términos de monitorización y control automatizado..

3. En este capítulo se presenta de forma integrada todo el ciclo de creación de la aplicación: desde los requerimientos que la guían, pasando por la metodología escogida para su desarrollo, hasta el diseño arquitectónico y de interfaz que sustenta su implementación. Primero se definen las capacidades (funcionales y no funcionales) que el sistema debe ofrecer. A continuación, se describe el enfoque de trabajo —iterativo e incremental—, las herramientas de gestión empleadas y la planificación temporal de las distintas fases. Finalmente, se despliega el diseño del sistema: su arquitectura de componentes y los prototipos de la interfaz gráfica, ilustrando cómo se han traducido los requisitos en una solución técnica coherente.
4. Se aborda la parte práctica del desarrollo, describiendo paso a paso la integración y configuración de las tecnologías elegidas. Se comentan decisiones de implementación relevantes, la estructuración del código y cualquier patrón de diseño empleado.
5. En este capítulo se especifican los planes de prueba diseñados para asegurar la calidad del producto. Estarán incluidos las pruebas unitarias, de integración y las posibles pruebas de campo. Además se expondrán los resultados obtenidos y las validación de los objetivos cumplidos.
6. Aquí se presentan y analizan los resultados del proyecto, discutiendo su efectividad y reflexionando sobre las limitaciones encontradas y la interpretación de los datos conseguidos.
7. Este último capítulo recapitula los logros del trabajo relacionándolos con los objetivos planteados. También se sugieren nuevas líneas de investigación o mejoras posibles que no se han podido llevar a cabo en este Trabajo de Fin de Grado.

Estado del Arte y Antecedentes

El presente capítulo tiene como propósito dar un marco conceptual y tecnológico claro que permita comprender en profundidad la tecnología utilizada en este Trabajo de Fin de Grado, así como los antecedentes y contextos donde se aplica dicha tecnología.

En primer lugar se describirán las características esenciales del metano, destacando su importancia ambiental e industrial debido a su gran impacto en el calentamiento global. Posteriormente, se introducirá la tecnología TDLAS (Tunable Diode Laser Absorption Spectroscopy), explicando sus fundamentos físicos y sus ventajas y limitaciones para la detección y monitorización precisa de gases.

Además, se aborda una revisión de algunas soluciones existentes para la medición del metano, tanto sistemas estacionarios como móviles, incluyendo aquellos que integran drones y software especializado. Finalmente, se exponen herramientas complementarias como ROS, destacando su relevancia para la automatización y coordinación en procesos industriales y ambientales. Este conjunto de elementos permite contextualizar adecuadamente la relevancia y alcance del proyecto presentado.

2.1. Fundamentos del Metano

2.1.1. Características del Metano

El metano (CH_4) es un hidrocarburo alifático sencillo, compuesto por un átomo de carbono unido a cuatro átomos de hidrógeno. Se presenta en condiciones estándar como un gas incoloro, inodoro e inflamable, siendo el hidrocarburo más simple y uno de los gases de efecto invernadero más abundante en nuestra atmósfera. Tiene una densidad menor que el aire y

como hemos dicho anteriormente, es altamente inflamable cuando se mezcla con oxígeno en determinadas proporciones. El metano tiene una capacidad de absorción de radiación infrarroja mayor que la de dióxido de carbono, lo cuál incrementa en gran medida su efecto en el calentamiento global [6].

2.1.2. Aplicaciones Industriales y Ambientales

El metano se encuentra principalmente en fuentes naturales como humedales, procesos de fermentación anaerobia y también en actividades antropogénicas como la explotación de hidrocarburos, gestión de residuos orgánicos y agricultura intensiva. Su monitorización es esencial en sectores industriales como la energía, agricultura, tratamiento de residuos y minería, debido a sus implicaciones ambientales y su potencial energético como combustible fósil en la generación de energía eléctrica y térmica [9].

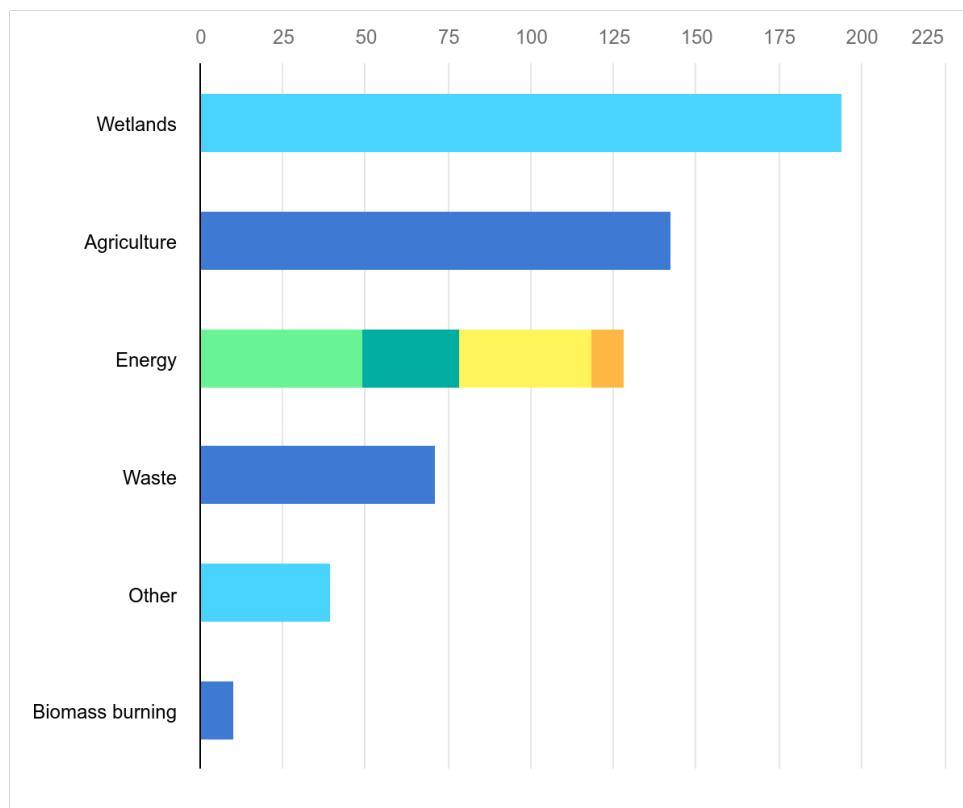


Figura 1: Fuente de las emisiones de metano en el año 2023 [3]

2.2. Introducción a la Tecnología TDLAS

2.2.1. Fundamentos físicos de TDLAS

La espectroscopia de absorción con láser de diodo sintonizable (TDLAS, por sus siglas en inglés) es una poderosa técnica analítica ampliamente utilizada para detectar y medir concentraciones de gases a longitudes de onda específicas. Este método emplea láseres de diodo sintonizables para sondear líneas de absorción específicas de moléculas de gas, permitiendo la identificación y cuantificación precisa de especies gaseosas en mezclas complejas. La técnica se basa en la absorción de la luz láser por moléculas de gas. Cuando un láser de diodo sintonizable emite luz a una longitud de onda específica que corresponde a una línea de absorción de una molécula de gas, el gas absorbe una parte de la luz. Si se sintoniza el láser a lo largo de la línea de absorción, se puede medir la cantidad de luz que ha sido absorbida, lo que permite determinar la concentración del gas presente en el medio. La ley de Beer-Lambert describe la relación entre la absorbancia y la concentración de las especies absorbentes:

$$A = \log\left(\frac{I_0}{I}\right) = \epsilon \cdot c \cdot L$$

donde:

- A es la absorbancia.
- I_0 es la intensidad de la luz incidente.
- I es la intensidad de la luz transmitida.
- ϵ es el coeficiente de absorción molar.
- c es la concentración del gas.
- L es la longitud del camino óptico.

2.3. Soluciones existentes para la detección y monitorización de Metano

2.3.1. Sistemas de medición estacionarios

Los sistemas estacionarios de detección de metano están diseñados para ser instalados de forma fija en ubicaciones estratégicas dentro de las instalaciones industriales o zonas de almacenamiento de residuos como los vertederos. Estos dispositivos permiten realizar un monitoreo continuo y automatizado de las concentraciones de gases en el ambiente, generando alertas cuando superan cierto umbral predefinido. Normalmente, utilizan tecnologías como sensores infrarrojos no dispersivos (NDIR), espectroscopía de absorción o sensores catalíticos. Una de sus principales ventajas es la fiabilidad que nos ofrecen en entornos controlados, aunque presentan restricciones cuando se desea cubrir grandes áreas o acceder a zonas remotas, ya que su capacidad de supervisión se limita al entorno inmediato en el que están ubicados.

2.3.2. Sistemas móviles y uso de drones

Las soluciones móviles para la detección de metano incluyen tanto unidades terrestres como aéreas. En particular, el uso de drones equipados con sensores ligeros ha experimentado una gran expansión, permitiendo realizar inspecciones rápidas sobre infraestructuras distribuidas geográficamente. Estas plataformas aéreas son ideales para explorar zonas de difícil acceso, como campos agrícolas, terrenos elevados, etc. Al integrar tecnología como TDLAS, los drones pueden realizar mediciones precisas mientras sobrevuelan las zonas de interés. Sin embargo, factores como el viento, la altitud del vuelo o el tiempo de operación pueden influir negativamente en la precisión de las lecturas. Se requiere una planificación muy cuidadosa y operadores capacitados para maximizar su eficacia [2].

2.3.3. Soluciones de software y automatización existentes

El desarrollo de software ha permitido transformar el modo en que se gestionan y visualizan los datos capturados por los sensores. Muchas de las herramientas existentes están diseñadas para trabajar en tiempo real, integrándose con plataformas robóticas como ROS. Todo esto permite al operador obtener una visión clara del entorno mediante visualizacio-

nes gráficas, representaciones en 2D o 3D, e interfaces interactivas que simplifican el control del sistema. Aplicaciones como Rviz o FoxGlove Studio ofrecen entornos de trabajo flexibles que soportan múltiples flujos de datos, facilitando tareas como la calibración de sensores, el registro de mediciones o el análisis post-misión. Este tipo de soluciones contribuye a una automatización del proceso de detección, reduciendo la carga operativa y mejorando la capacidad de respuesta ante anomalías [4].

2.4. Herramientas y Tecnologías Complementarias

2.4.1. ROS (Robot Operating System)

ROS es un entorno de desarrollo flexible y potente utilizado para crear aplicaciones robóticas complejas y avanzadas. Su arquitectura basada en componentes facilita en gran medida la integración y comunicación entre los distintos tipos de dispositivos como pueden ser sensores, actuadores y unidades computacionales. ROS opera mediante un sistema de comunicación basado en tópicos, servicios y mensajes, permitiendo el intercambio fluido de información en tiempo real, esencial para aplicaciones críticas como la monitorización ambiental, navegación autónoma y la automatización industrial.

La popularidad y utilidad de ROS reside en la capacidad que tiene para gestionar sistemas distribuidos y en tiempo real, además de su naturaleza modular que permite la rápida integración y modificación de componentes individuales. Gracias a su extensa comunidad de usuarios, cuenta con un sinfín de nuevas herramientas, bibliotecas y soporte técnico, haciendo de ROS una opción preferida en el ámbito de la investigación y desarrollo robótico [4].

2.4.2. Interfaces gráficas y aplicaciones de control

Las interfaces gráficas juegan un rol crucial en sistemas robóticos y aplicaciones de monitorización complejas, ya que permiten gestionar visualmente grandes cantidades de datos y tomar decisiones rápidas y eficaces. Un ejemplo destacado es RViz, una herramienta que ofrece capacidades avanzadas de visualización en 3D, ideal para mostrar datos obtenidos desde múltiples sensores y realizar un seguimiento preciso del movimiento y posicionamiento de robots autónomos.

Además, Foxglove Studio proporciona una plataforma innovadora para la gestión y vi-

sualización detallada de múltiples flujos de datos. Esta herramienta permite visualizar simultáneamente diversos tipos de información, incluyendo mapas, gráficos y datos en tiempo real, simplificando significativamente tareas complejas como la configuración inicial del sistema, el análisis posterior a misiones o la resolución ágil de problemas operativos en campo.

2.5. Automatización y control de procesos de aplicaciones ambientales e industriales

2.5.1. Conceptos clave de automatización

La automatización consiste en la utilización de sistemas y tecnologías para controlar y monitorear procesos sin intervención humana constante, incrementando la eficiencia, precisión y seguridad operativa. Este concepto se basa en la integración de sistemas computacionales, sensores y actuadores que trabajan en conjunto para reducir la necesidad de control manual disminuyendo significativamente los errores humanos. En aplicaciones ambientales e industriales, la automatización es muy importante debido a la complejidad de estos procesos. Por ejemplo, en el monitoreo ambiental continuo de gases o contaminantes, la automatización no solo asegura una medición precisa y constante, sino que también facilita una respuesta rápida ante situaciones anómalas, optimizando la gestión del riesgo ambiental y mejorando considerablemente la seguridad operativa. La relevancia de estos sistemas reside en su capacidad para mantener un control continuo y efectivo, mejorando la calidad y confiabilidad de los resultados obtenidos.

2.5.2. Sistemas actuales de monitorización y control

Actualmente, los sistemas de monitorización y control se componen de múltiples elementos interconectados como sensores, controladores programables (PLC), interfaces gráficas, redes de comunicación y bases de datos, que permiten la adquisición, procesamiento y visualización efectiva de datos en tiempo real. Estos sistemas funcionan mediante la captura de datos del entorno, que luego son procesados y analizados automáticamente para proporcionar información relevante y en ocasiones activar respuestas automáticas según parámetros previamente definidos.

Su utilidad radica principalmente en mejorar la precisión y eficiencia operativa, permi-

tiendo detectar rápidamente fallas y tomar las medidas correctivas oportunas. Esto resulta especialmente útil en industrias como la energética y la gestión de residuos, donde el control constante y la rápida reacción ante posibles incidentes son críticos. La implementación adecuada de estos sistemas genera mejoras significativas en la seguridad operativa, la gestión de recursos y la reducción de costos operativos.

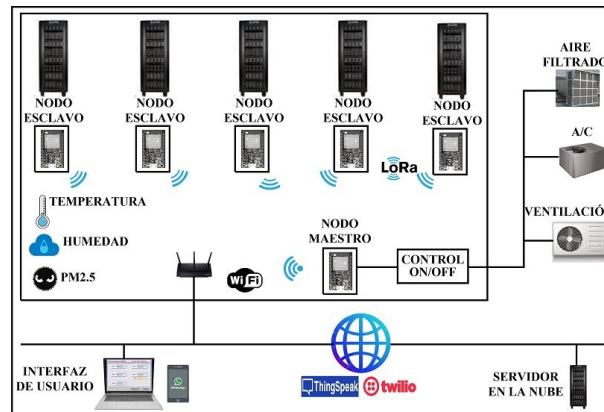


Figura 3: Ejemplo de un esquema de red de monitorización de un sistema automático de enfriamiento [1]

3

Proceso de Desarrollo y Diseño del Sistema

El éxito de un proyecto de software radica tanto en la claridad con la que se establecen sus requerimientos como en el seguimiento de una metodología de desarrollo y en la solidez de su diseño arquitectónico. En este capítulo reunimos estos tres pilares en una única visión unificada.

3.1. Requerimientos del sistema

El presente apartado ofrece el marco de requerimientos del sistema, definiendo los criterios operativos y técnicos que guiarán el diseño y desarrollo de la aplicación. En primer lugar, se describen los requerimientos funcionales, especificando las tareas y procesos que la herramienta debe ejecutar para supervisar, controlar y registrar las mediciones de metano en tiempo real. A continuación, se presentan los requerimientos no funcionales, estableciendo los estándares de calidad, rendimiento, seguridad y mantenibilidad necesarios para garantizar la eficacia y fiabilidad del sistema. Finalmente, se exponen las consideraciones técnica y límites operativos, detallando el entorno de ejecución, formatos de comunicación y restricciones de alcance del sistema.

3.1.1. Requerimientos funcionales

Los requerimientos funcionales son aquellos que describen las funciones y características específicas que el sistema debe cumplir. Estos requerimientos son esenciales para garantizar

que la aplicación cumpla con los objetivos establecidos y satisfaga las necesidades de los usuarios.

A continuación, se presentan los requerimientos funcionales identificados para la aplicación de supervisión, control y registro de mediciones de metano:

1. Configuración del Experimento

La aplicación debe gestionar a la perfección la fase de la configuración del experimento, es decir, inicialización de las variables y configuración de los dispositivos involucrados.

2. Gestión de Dispositivos

La aplicación deberá gestionar de manera centralizada la comunicación con todos los dispositivos involucrados (sensores, unidades de posicionamiento y plataformas móviles), mostrando de forma clara y actualizada su estado operativo, nivel de batería (si es posible) y posibles errores. Además, deberá integrar la información de posición en el mapa y permitir el envío de comandos básicos (como ajustes de trayectoria o parámetros de funcionamiento) sin interrumpir la operación global.

3. Monitorización en Tiempo Real

Debe capturar y representar datos instantáneos de metano en ppm (partes por millón), lecturas auxiliares, así como la posición geográfica de cada dispositivo sobre un mapa interactivo. La visualización de los datos de concentración de metano debe realizarse directamente sobre el mapa mediante la representación de rayos dinámicos que actualizarán su intensidad y alcance en función de las mediciones obtenidas. Todo esto junto a la existencia de paneles de estado que muestren la información de cada dispositivo, permitirán al usuario tener una visión clara y precisa del estado del experimento en todo momento, pudiendo tomar decisiones informadas y rápidas en caso de que sea necesario.

4. Delimitación del Área de Inspección

El usuario podrá dibujar polígonos o círculos sobre un mapa 2D para señalar las áreas por las que debe desplazar el robot. La aplicación debe permitir la fácil edición y eliminación de estas delimitaciones, asegurando que el usuario pueda ajustar la zona de inspección según sea necesario. Además, se debe permitir el almacenamiento de estas delimitaciones para su uso en experimentos futuros, facilitando la reutilización de configuraciones previas.

5. Control Operativo Dinámico

En cualquier momento del experimento, el operador debe poder pausar, abortar o reiniciar la sesión de medición. Así, el operador tiene la capacidad de tomar decisiones rápidas y efectivas en función de los datos obtenidos y del estado del experimento.

6. Registro de Datos

Todas las mediciones realizadas, relacionadas con la posición de los distintos dispositivos, deberán almacenarse en un archivo rosbag para su posterior simulación y análisis. Este archivo debe incluir tanto los datos de metano como las lecturas auxiliares, así como la información de posición de cada dispositivo. Además, se debe permitir la exportación de estos datos a otros formatos comunes (CSV, JSON, etc.) para facilitar su análisis posterior.

7. Simulación de Experimentos Registrados

La aplicación deberá permitir la reproducción de los experimentos registrados en el archivo rosbag. Durante la simulación, el usuario podrá ejecutar acciones como pausar, reproducir a diferentes velocidades, avanzar de mensaje a mensaje. Además, se debe permitir la visualización de los datos de concentración y las posición de los dispositivos sobre el mapa de manera sincronizada, pudiendo replicar el experimento en un entorno controlado.

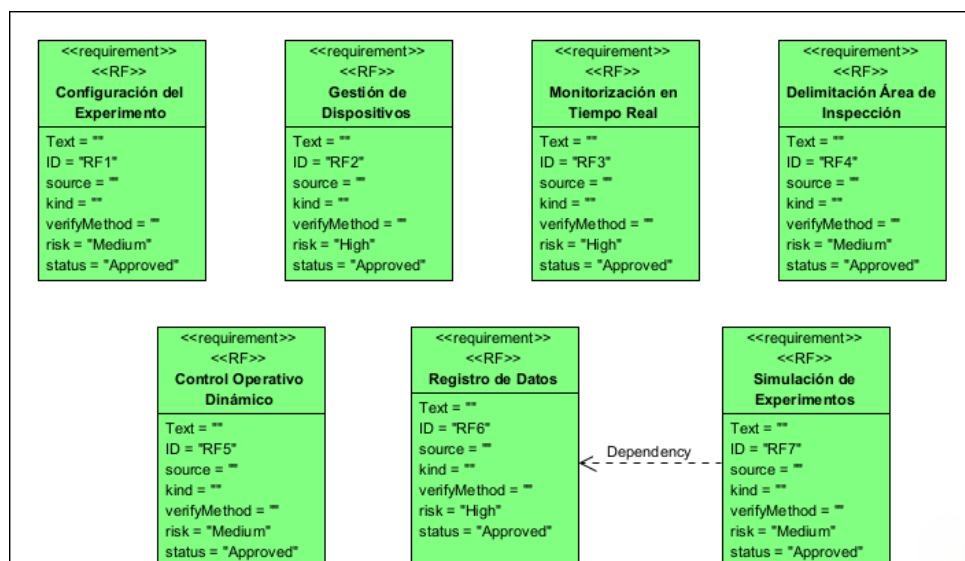


Figura 4: Diagrama de flujo de los requerimientos funcionales.

3.1.2. Requerimientos no funcionales

Los requerimientos no funcionales son aquellos que describen las características y cualidades del sistema, como su rendimiento, usabilidad, seguridad y mantenibilidad. Estos requerimientos son igualmente importantes para garantizar la calidad y eficacia de la aplicación. A continuación, se presentan los requerimientos no funcionales identificados para la aplicación:

1. Usabilidad

La interfaz gráfica de la aplicación debe ser intuitiva y fácil de usar, permitiendo a los usuarios navegar y operar el sistema sin dificultad. Se deben seguir principios de diseño centrado en el usuario para garantizar una experiencia fluida y eficiente. Además, se deberá proporcionar una documentación clara y accesible que explique el funcionamiento de la aplicación y sus características principales.

2. Rendimiento

La aplicación debe ser capaz de procesar y visualizar datos en tiempo real sin retrasos significativos. Esto incluye la captura, procesamiento y representación de datos de metano y lecturas auxiliares, así como la actualización del mapa interactivo.

3. Mantenibilidad

El código fuente de la aplicación debe estar bien estructurado y documentado, facilitando su mantenimiento y evolución a lo largo del tiempo. Se deben seguir buenas prácticas de programación y utilizar patrones de diseño adecuados para asegurar la calidad del código.

4. Compatibilidad

La aplicación debe ser compatible con diferentes sistemas operativos (Windows, Linux) y versiones de ROS, asegurando su funcionamiento en una amplia variedad de entornos.

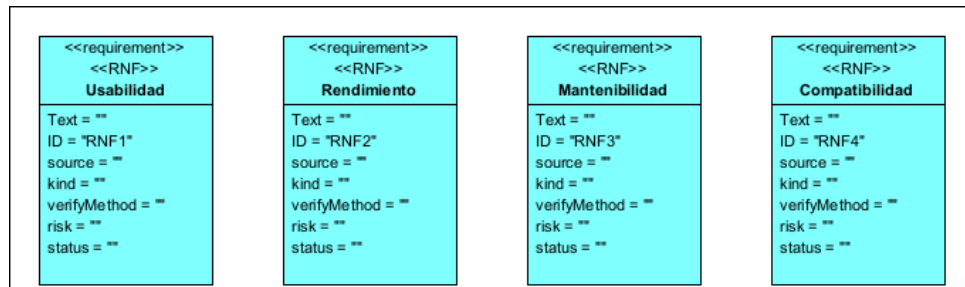


Figura 5: Diagrama de flujo de los requerimientos no funcionales.

3.2. Metodología de Desarrollo

La elección de una metodología adecuada resulta crucial para el éxito de cualquier proyecto de desarrollo de software, ya que orienta la organización del trabajo, la gestión de recursos y la adaptabilidad ante posibles cambios. En el contexto de este proyecto, se ha optado por un enfoque de trabajo **iterativo e incremental** debido a su capacidad para gestionar la complejidad y minimizar riesgos mediante entregas parciales continuas y validables. Este modelo permite corregir desviaciones de manera temprana y continua, incorporar retroalimentación constante y ajustar los requisitos de acuerdo a nuevas necesidades detectadas. A continuación, se detallan el enfoque metodológico seguido, las herramientas de gestión utilizadas y la planificación temporal de las distintas fases del proyecto.

3.2.1. Enfoque de trabajo: Desarrollo Iterativo e Incremental

El desarrollo de la aplicación se ha llevado a cabo siguiendo un enfoque iterativo e incremental que combinará los beneficios de los enfoques ágiles con la estructura del desarrollo tradicional. Esta elección se fundamentó en la necesidad de adaptarse a un entorno cambiante y a un alto nivel de complejidad técnica.

- **Iterativo:** Cada ciclo de desarrollo incluía una revisión completa de análisis, diseño, implementación y pruebas, lo que permitió pulir los requisitos, optimizar diseños y corregir desviaciones de forma continua.
- **Incremental:** El sistema se fue ampliando gradualmente mediante la adición de módulos autónomos que integraban nuevas funcionalidades, asegurando que cada versión fuera plenamente operativa y validable.

Gracias a este enfoque se logró:

- Minimizar el impacto de errores o cambios imprevistos al dividir el trabajo en entregas de menor tamaño y riesgo.
- Mejorar progresivamente la calidad de los componentes a través de retroalimentación frecuente y validación de requisitos.
- Alinear mejor los resultados con las necesidades reales del proyecto, adaptando las prioridades conforme evolucionaba el sistema.

3.2.2. Herramientas de Gestión Empleadas

Las herramientas de gestión son fundamentales para el seguimiento y control del proyecto, facilitando la planificación, la asignación de tareas y la comunicación entre los miembros del equipo. En este proyecto se han utilizado las siguientes herramientas:

- **Trello:** Utilizado como herramienta principal de organización de tareas mediante tableros Kanban, facilitando la visualización del flujo de trabajo. Cada tarea pasaba por columnas de "Pendiente", ^{En} Progreso y "Finalizada", lo que permitía monitorizar el avance del proyecto en tiempo real y reasignar prioridades de manera ágil. Trello permitió dividir el proyecto en módulos funcionales específicos, asignar tareas individuales a cada componente del sistema y mantener una visión global del avance.

Foto de Trello con el tablero de tareas del proyecto.

- **Git y GitHub:** Git es un sistema de control de versiones distribuido que permite gestionar los cambios en el código fuente de manera eficiente, facilitando el trabajo en equipo y asegurando la integridad del historial de desarrollo. GitHub, por su parte, es una plataforma basada en la nube que permite alojar repositorios Git y coordinar el trabajo colaborativo mediante herramientas adicionales como pull requests, revisiones de código, etc. Se emplearon en este proyecto para mantener un control de todas las modificaciones realizadas en el código. La elección de Git y GitHub se debió a su robustez, su amplio soporte en la comunidad de desarrolladores y su facilidad para gestionar flujos de trabajo colaborativos incluso en entornos de alta complejidad.

Foto de GitHub con el repositorio del proyecto.

- **Slack:** Slack es una plataforma de comunicación y colaboración en tiempo real diseñada para facilitar el trabajo en equipo mediante mensajes organizados en canales temáticos. En este proyecto, Slack fue utilizado como herramienta principal de comunicación interna, permitiendo la coordinación diaria entre el equipo docente y yo. La elección de Slack se justificó por su facilidad de integración con herramientas como Github y Trello, su interfaz intuitiva y su capacidad de mejorar la eficiencia de la comunicación, reduciendo los tiempos de respuesta y facilitando la colaboración en tiempo real.

3.2.3. Planificación Temporal

La planificación del proyecto se dividió en fases sucesivas, cada una con objetivos concretos:

Fase	Actividades principales
Análisis	Reunir requisitos, identificar actores y priorizar funcionalidades.
Diseño	Diagramas UML, diseño de bocetos y definición de la arquitectura.
Implementación	Configurar el experimento, registrar mediciones y visualizar datos.
Integración	Control dinámico y simulación de experimentos.
Pruebas	Validación mediante pruebas unitarias y de integración.
Documentación	Preparación de manual y documentación técnica.

Cuadro 1: Planificación temporal de fases del proyecto

Cada fase cerraba con una revisión de los objetivos alcanzados, detectando posibles desviaciones y replanificando de ser necesario. Esta planificación flexible aseguró que el desarrollo se mantuviera alineado con los requisitos y objetivos del proyecto, permitiendo adaptaciones rápidas ante cambios o imprevistos.

3.3. Diseño del Sistema

Transformar los requisitos iniciales en un diseño técnico coherente constituye uno de los pilares fundamentales en el desarrollo de cualquier sistema de software. Esta fase de diseño asegura que los requisitos funcionales y no funcionales no permanezcan como simples declaraciones teóricas, si no que se conviertan en arquitecturas e interfaces tangibles que guiarán la implementación del sistema. Así, el diseño actúa como un puente entre el análisis conceptual y la realidad operativa del sistema, asegurando coherencia, escalabilidad y mantenibilidad en el producto final.

3.3.1. Arquitectura de Componentes

La arquitectura del sistema desarrollado se basa en un diseño centralizado implementado dentro de un único nodo ROS, el cual centraliza múltiples funcionalidades para una gestión eficiente y coordinada del experimento de medición de metano. Este nodo está organizado internamente mediante módulos específicos, claramente separados en controladores y vistas, asegurando una lógica estructurada y modular. ⁶

Estructura Interna del Nodo

El nodo principal se encuentra en una estructura modular dividida en dos grandes bloques: controladores (lógica interna y gestión de los distintos dispositivos) y vistas (interfaz gráfica e interacción con el usuario). Esta separación permite una clara distinción entre la lógica de negocio y la presentación, facilitando la mantenibilidad y escalabilidad del sistema.

Entre los controladores nos encontramos:

- **Main Controller** (`main_controller.py`): Actúa como el controlador central del sistema, que inicializa el resto de los subcontroladores (robot, TDLAS y PTU), conecta sus señales y gestiona su integración con la interfaz gráfica. Es el que se encarga de coordinar todos los eventos que ocurren en el sistema, publicar parámetros de configuración, registrar y reproducir los experimentos realizados a través de datos guardados en rosbag, y controlar el estado global del experimento. Además, centraliza la lógica de comprobación del estado de los dispositivos y gestiona el flujo de ejecución del sistema,

desde la configuración, pasando por la ejecución y finalización del experimento, hasta la simulación y almacenamiento de los datos obtenidos.

- **PTU Controller** (`ptu_controller.py`): Este controlador maneja específicamente el control de la unidad de posicionamiento (PTU). Se encarga de actualizar la posición de la PTU, gestionar su estado de preparación y visualizar su posición en la interfaz gráfica. Además, proporciona funcionalidades para facilitar la configuración interactiva del dispositivo mediante la interfaz gráfica.
- **Robot Controller** (`robot_controller.py`): Gestiona el control de plataformas móviles, en este caso el robot denominado Hunter, del laboratorio de investigación MAPIR. Además, es el encargado tanto de la actualización de parámetros como la velocidad y la posición, como de la configuración dinámica de la trayectoria que debe seguir el robot. Este controlador actúa directamente con la interfaz gráfica para mostrar el estado del robot y facilitar su configuración mediante pantallas específicas.
- **TDLAS Controller** (`tdlas_controller.py`): Controla la disponibilidad y el estado del sensor TDLAS, actualizándolo en la interfaz gráfica a través de señales propias de PyQt, garantizando que el sistema lo reconozca y lo integre correctamente en el flujo de trabajo.

Las vistas, por su parte, implementan una interfaz gráfica amigable utilizando la librería PyQt5, facilitando así la interacción del usuario con el sistema a través de componentes visuales:

- **Main Window** (`main_window.py`): Esta vista es la ventana principal de la aplicación, desde la cuál se centraliza la visualización, navegación y configuración de los distintos módulos del sistema. Permite cambiar entre las diferentes vistas de la aplicación, ajustar dinámicamente el tamaño del mapa según la resolución de pantalla y lanzar los diálogos de la PTU, el robot móvil y la selección de trayectorias de manera modular e integrada.
- **Splash Screen** (`splash_screen.py`): Pantalla de arranque que se muestra al iniciar la aplicación, proporcionando una experiencia de usuario más atractiva y profesional. Esta pantalla incluye el logotipo de la aplicación y te muestra aquellos nodos que quedan

por cargar junto con los que ya están cargados. Además, permite al usuario visualizar el progreso de la carga de los distintos módulos del sistema, asegurando que el usuario esté informado sobre el estado de la aplicación antes de que esté completamente operativa.

- **Componentes visuales:** La interfaz gráfica incluye componentes personalizados como los siguientes:
 - **Device Card** (`device_card.py`): Un widget personalizado que contiene información detallada del estado operativo de los dispositivos individuales.
 - **SatelliteMap** (`map_view.py`): Representación dinámica y visual en tiempo real de los datos geográficos y mediciones sobre un mapa interactivo. Incluye todas aquellas funciones necesarias para conectar el widget de PyQt con la visualización de Google Maps a través de HTML y JavaScript.
 - **Title Bar** (`title_bar.py`): Barra superior de la ventana principal que incluye que incluye información sobre la aplicación y una barra de navegación para acceder a las distintas vistas principales.
 - **Toast** (`toast.py`): Un componente visual que muestra mensajes de estado y notificaciones al usuario, permitiendo una comunicación clara y efectiva de eventos importantes o errores en el sistema.
- **Home Page** (`home_page.py`): Esta vista es la página de inicio de la aplicación, desde donde se accede a las distintas funcionalidades del sistema. Incluye botones de acceso rápido a las vistas de configuración del robot, la PTU y la selección de trayectorias, así como un mapa interactivo que muestra tanto la posición de los dispositivos como la medición del metano en tiempo real.
- **PTU Config** (`ptu_config.py`) y **Robot Config** (`robot_config.py`): Permiten configurar los parámetros específicos para los dispositivos individuales, en este caso la PTU y el robot móvil.
- **Simulation Page** (`simulation_page.py`): Esta vista permite al usuario simular los experimentos registrados en el archivo rosbag, proporcionando una interfaz para la reproducción de los datos y la visualización de las mediciones en tiempo real. Incluye contro-

les para pausar, avanzar y abortar en la simulación, así como opciones para ajustar la velocidad de reproducción.

- **TrajectorySelectorDialog** (select_location_dialog.py): Permite al usuario seleccionar, visualizar, renombrar o eliminar trayectorias guardadas para el experimento, integrando vistas previas interactivas en tarjetas y emitiendo señales según la selección realizada.

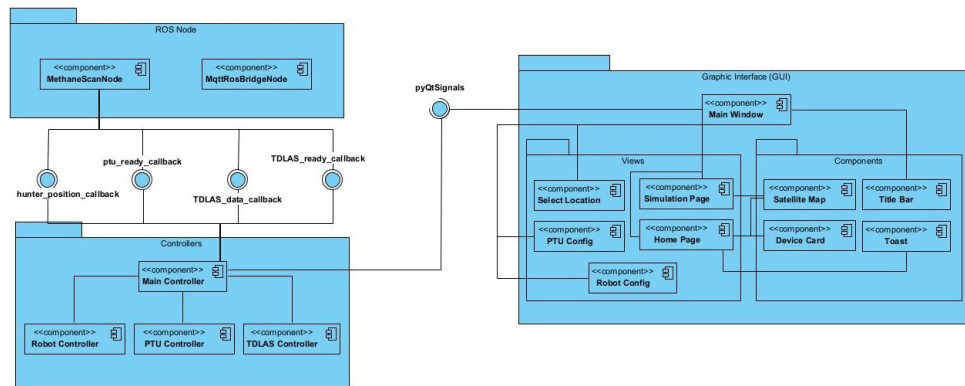


Figura 6: Diagrama de componentes del sistema.

Comunicación Interna del Nodo

Aunque el sistema está estructurado mayoritariamente en torno a un único nodo central que coordina la lógica del experimento, su arquitectura ROS2 refleja una integración sofisticada entre procesamiento en segundo plano, manejo de eventos y una interfaz gráfica interactiva mediante PyQt. Este nodo, actúa como núcleo de control, gestionando múltiples suscripciones y publicaciones a tópicos ROS2, así como la emisión de señales **Qt Thread-Safe** que permiten la sincronización lógica de la interfaz.

La comunicación no se limita a una transmisión de mensajes simple, si no que se apoya en una arquitectura basada en señales y slots propios de la librería PyQt. Las distintas señales son transmitidas tanto desde hilos ROS2 como de los mismos componentes Qt, siendo procesadas de forma segura en el hilo principal de la aplicación. Esto garantiza la integridad de la interfaz de usuario y la respuesta oportuna antes eventos del sistema, como cambios en el estado de los dispositivos o la recepción de datos de los mismos.

Además, el nodo permite registrar callbacks personalizados para cada tipo de dato recibido. Esta estrategia de diseño modular y desacoplada permite que, al recibir datos de los distintos

dispositivos, se ejecuten funciones específicas en los controladores correspondientes sin bloquear la ejecución general del sistema.

El nodo también implementa mecanismos seguros para pausar o reanudar suscripciones, lo cual es especialmente útil durante la tarea de simulación de experimentos.

En la interfaz gráfica, los distintos controladores actúan como intermediarios entre el nodo y las vistas. Estos controladores reciben los datos del nodo a través de señales y callbacks, procesan la información y actualizan la interfaz gráfica mediante diversos métodos. Por ejemplo, cuando se reciben datos del dispositivo TDLAS, el controlador principal los procesa y actualiza tanto la visualización en el mapa como las etiquetas de datos en la interfaz.

En definitiva, el nodo central no solo publica y suscribe información como los niveles de metano, posición del robot o estados de los dispositivos, sino que además actúa como puente robusto entre ROS2 y PyQt. Esto garantiza que los eventos se gestionen de forma asincrónica, eficiente y segura dentro de la aplicación global de medición de metano, permitiendo una interacción fluida entre los sensores, la lógica del sistema y la interfaz gráfica.

3.3.2. Arquitectura de Comunicaciones (RSOS2 <->MQTT <->Robots)

El sistema desarrollado no se limita a una arquitectura ROS2 tradicional, sino que se complementa con una infraestructura de mensajería basada en **MQTT**, lo que permite habilitar un canal de comunicación ligero, interoperable y orientado a la integración con plataformas remotas. Esta integración se articula a través de un nodo intermedio denominado `mqttr_ros_bridge_node`, como se observa en la figura 6. Este nodo actúa como puente entre el ecosistema local de ROS2 y un servidor MQTT al que se conectan dispositivos clientes.

Introducción a MQTT

MQTT (Message Queuing Telemetry Transport) es un protocolo de mensajería ligero y eficiente diseñado para la comunicación entre dispositivos, basado en el modelo publicación-suscripción, al igual que ROS2. Este protocolo está diseñado para comunicaciones máquina a máquina (M2M) y es especialmente útil en entornos donde la conectividad es limitada o inestable. Es ampliamente utilizado en aplicaciones de IoT (Internet de las Cosas) y sistemas embebidos, donde la eficiencia y la baja sobrecarga son cruciales.

En esta arquitectura, MQTT se utiliza como medio de comunicación entre el sistema ROS2

local y los dispositivos individuales remotos. Esto permite, por ejemplo, enviar la trayectoria que debe seguir el robot móvil o recibir datos de sensores externos.

Un componente clave de MQTT es el *broker*, que actúa como intermediario entre los clientes. Es el encargado de recibir todos los mensajes publicados y reenviarlos a los clientes suscritos a los tópicos correspondientes.

Esquema General de Comunicación

La arquitectura desarrollada se organiza en tres niveles principales de comunicación:

1. ROS2 Interno (nodo `methane_scan_node`):

Es el núcleo principal del sistema de control local. Gestiona de forma centralizada la lógica del experimento, donde se incluyen la operación de los distintos dispositivos. Además, coordina la interfaz gráfica desarrollada en PyQt5, permitiendo la interacción del usuario con el sistema.

Otra de sus funciones es la de emitir mensajes estructurados en formato JSON hacia tópicos ROS2 internos, representado comando de ejecución o de configuración de los dispositivos. Estos mensajes son generados por el usuario a través de la interfaz gráfica y enviados al nodo.

Por último, el nodo también se encarga de escuchar y recibir los mensajes provenientes de sistemas externos, canalizados a través de MQTT, los cuales llegan convertidos desde el nodo puente, `mqtt_ros_bridge_node`, y permiten actualizar el estado del sistema o reaccionar ante eventos externos.

2. Nodo Intermedio (puente `mqtt_ros_bridge_node`):

En general, este nodo actúa como un traductor entre ROS2 y MQTT, permitiendo la comunicación entre el sistema local y dispositivos remotos. Este nodo recibe mensajes ROS2 y los encapsula en objetos de tipo **KeyValue**. Este objeto está compuesto por una clave y un valor, donde la clave representa el tópico MQTT al que se publicará el mensaje y el valor es el contenido del mensaje en sí. El mensaje se publica en el tópico `/ros2mqtt` para que lleguen al cliente MQTT.

Por otro lado, el nodo se suscribe a `/mqtt2ros`, donde recibe objetos **KeyValue** desde el cliente MQTT. Se interpreta la clave del objeto como el tópico ROS2 al que se publicará el mensaje y el valor como el contenido del mensaje, reconvirtiéndolo a su tipo original

(Bool, String, etc.).

3. Cliente MQTT (nodo `mqtt_bridge`):

Es el que interactúa directamente con el broker MQTT. Se suscribe a `/ros2mqtt` para recibir mensajes ROS encapsulados, que luego redirige al dispositivo físico o embebido. Este nodo también se encargará de publicar mensajes en el tópico `/mqtt2ros` cuando un dispositivo externo genera una respuesta o señal de estado, encapsulando de nuevo el mensaje en un objeto **KeyValue** para que el nodo puente lo convierta a su tipo original y lo publique en el tópico ROS2 correspondiente.

El flujo de mensajes entre los distintos nodos se representa en la figura 7.

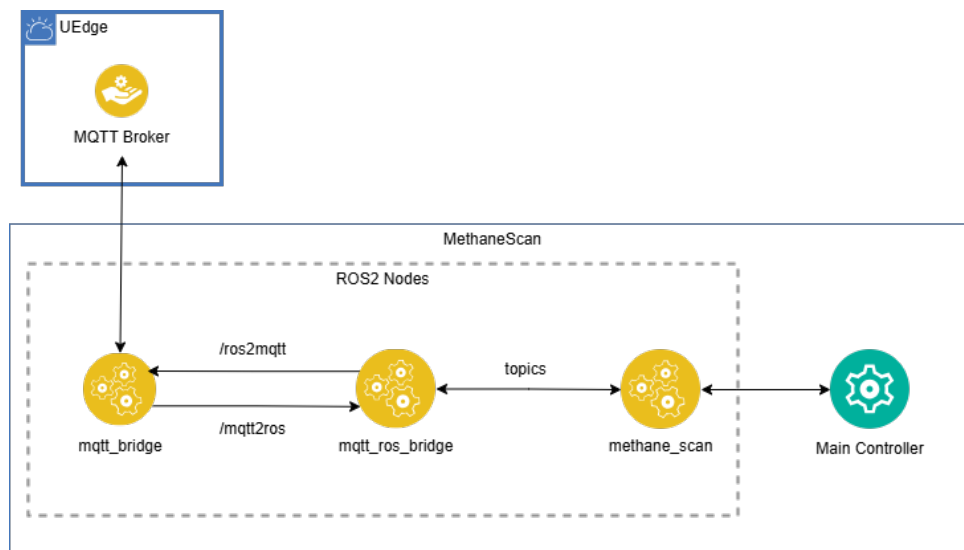


Figura 7: Diagrama de flujo de la arquitectura de comunicaciones.

Ventajas de la Arquitectura elegida

La elección de esta arquitectura de comunicaciones basada en MQTT y ROS2 ofrece varias ventajas significativas:

- **Interoperabilidad:** La arquitectura permite integrar ROS2 con aquellos sistemas heterogéneos que no manejen ROS directamente, facilitando la comunicación entre dispositivos de diferentes fabricantes o tecnologías.
- **Modularidad:** Al delegar la comunicación a un nodo puente, se logra una separación clara entre el sistema local y los dispositivos remotos. Esto permite una mayor flexibi-

lidad en la implementación y gestión de los dispositivos, facilitando su configuración y control.

- **Tolerancia a fallos:** MQTT es tolerante a fallos de red y permite la reconexión automática de los clientes. Esto significa que, incluso en entornos con conectividad intermitente, el sistema puede seguir funcionando sin interrupciones significativas.
- **Estandarización:** El uso del tipo de dato **KeyValue** para encapsular los mensajes facilita un protocolo de comunicación flexible y comprensible desde cualquier cliente MQTT.
- **Escalabilidad:** La arquitectura permite añadir nuevos dispositivos o sensores de manera sencilla, sin necesidad de modificar la lógica del sistema central. Esto facilita la expansión del sistema a medida que se incorporan nuevos elementos.

3.3.3. Patrón de diseño empleado

El patrón de diseño que se ha utilizado para el desarrollo del sistema es, como se ha podido intuir anteriormente, el Modelo-Vista-Controlador (MVC). Este patrón organiza el software en tres componentes principales claramente diferenciados, permitiendo así una arquitectura robusta y fácilmente mantenible.

- **Modelo::** En el contexto de este proyecto y dado el uso específico de ROS2, el "modelo" como tal no está implementado de manera explícita como un módulo separado. En su lugar tenemos el nodo de ROS2, que se encarga directamente de gestionar y recibir todos los datos provenientes de los distintos dispositivos mediante suscripciones a tópicos. Estos datos recibidos se entregan directamente al controlador correspondiente, que se encarga de procesarlos según la lógica del experimento. Por lo tanto, aunque no existe un módulo específico denominado "modelo", el nodo realiza una función similar, actuando como un repositorio y canalizador de la información hacia el controlador.
- **Vista:** Se centra exclusivamente en la presentación visual y en facilitar la interacción del usuario con el sistema mediante una interfaz gráfica intuitiva desarrollada con PyQt5. La vista no realiza ningún procesamiento ni posee conocimiento de la lógica del sistema. Su función es únicamente mostrar el estado actual de la aplicación al usuario en tiempo

real, permitiendo la visualización de los datos, mostrar los mensajes de estado o alertas y de recopilar y canalizar las acciones del usuario hacia el controlador.

- **Controlador:** Cumple el el papel de intermediario entre el nodo ROS2 (que en este caso actúa como fuente directa de datos) y la vista. Este componente recibe las interacciones del usuario a través de la vista, procesa los datos recibidos del nodo y actualiza adecuadamente el sistema según la lógica de negocio establecida. Además, responde a cualquier cambio en los datos recibidos, asegurando que estos datos se reflejen de manera rápida y precisa en la vista. Es el encargado de recibir los eventos del usuario y traducirlos en acciones concretas, manteniendo la coherencia y la sincronización entre el modelo y la vista.

Razones para la elección del patrón MVC

La elección del patrón MVC en este proyecto se fundamenta en diversas consideraciones técnicas y prácticas. En primer luha, destacamos la clara separación de responsabilidades que ofrece, permitiendo simplificar en gran medida tanto el desarrollo inicial como el mantenimiento futuro del sistema. Al diferenciar claramente entre la lógica del sistema, la lógica de presentación y la gestión de datos, es más fácil aislar y resolver errores, y permitir una expansión más ordenada y eficiente.

Por otro lado, aunque en este proyecto específico el desarrollo lo ha realizado una sola persona, el patrón MVC facilita en gran medida la organización y gestión del trabajo al permitir una separación clara de las distintas tareas y funcionalidades. Si en un futuro se decidiera ampliar el equipo de desarrollo, la estructura modular del patrón MVC facilitaría la incorporación de nuevos desarrolladores, permitiendo que cada uno se enfoque en una parte específica del sistema sin interferir en el trabajo de los demás.

Otra razón importante es su modularidad, que permite reutilizar componentes específicos en otros proyectos similares, mejorando así la productividad y reduciendo el tiempo en futuros desarrollos.

Finalmente, el MVC es adecuado en contextos en los que la interfaz de usuario necesita reflejar rápidamente cambios dinámicos en los datos, garantizando así una interacción fluida y efectiva con el usuarios final.

3.3.4. Modelo de Datos

El modelo de datos del sistema está diseñado para asegurar una gestión eficiente, estructurada y persistente de toda la información generada y utilizada durante la ejecución del experimento y su posterior simulación y análisis. Este modelo abarca dos tipos fundamentales: el almacenamiento de los mensajes enviados por tópicos durante la ejecución del experimento mediante el sistema ROSbag, y la gestión persistente de trayectorias mediante archivos serializados con la librería **Pickle** de Python.

Estructura de almacenamiento en ROSbag

ROSBag es una herramienta proporcionada por ROS2 que permite almacenar mensajes publicados en tópicos específicos durante la ejecución de experimentos en formato binario comprimido. Este formato es altamente eficiente y facilita enormemente la recuperación posterior y la reproducción exacta de condiciones experimentales anteriores para su análisis o validación mediante simulaciones.

En este sistema, cada experimento realizado se registra en un archivo ROSbag separado, que incluye todas las variables relevantes emitidas en tiempo real por los distintos dispositivos del experimento, como los robots o el medidor de metano. La organización de los archivos ROSbag es secuencial y estructurada en función de la fecha y hora en la que se realizó, permitiendo una fácil identificación y recuperación de los datos.

Variables registradas en el archivo ROSbag

Para el correcto registro de las variables, se ha definido un tópico local (`/save_simulation`), el cuál no tiene suscriptores, para compactar todos los mensajes en un solo tópico, sea más fácil su almacenamiento y la coordinación de mensajes sea más rápida y efectiva. Las principales variables registradas en el archivo ROSbag durante los experimentos son las siguientes:

- **Posición del Robot:** Incluye la posición del robot móvil en cada instante, dividida en latitud y longitud.
- **Estado de la PTU:** Registra la posición de la PTU en cada instante, al igual que el robot.
- **Mediciones de Metano:** Incluye los niveles de metano medidos por el sensor TDLAS en cada instante. Está compuesto por un diccionario con la siguiente estructura:

- *average_ppmxm*: Promedio de la medición de metano en partes por millón.
- *average_reflection_strength*: Promedio de la fuerza de reflexión del láser.
- *average_absorption_strength*: Promedio de la fuerza de absorción del láser.
- *header*: Información adicional sobre la medición, como el timestamp, dividido en nanosegundos y segundos.

Estos datos se almacenan en formatos nativos de ROS2(JSON, String, Bool, etc.), que posteriormente son guardados en archivos ROSbag en formato binario comprimido. Este formato permite una captuea eficiente y ordenada del flujo de datos en tiempo real, ocupando menos espacio en el disco y facilitando su manipulación dentro del ecosistema de ROS. Durante la fase de simulación, los datos almacenados recuperan automáticamente y reconstruyen su tipo original, permitiendo que las variables se comporten exactamente igual que durante la ejecución en vivo.

Almacenamiento y reutilización de trayectorias mediante Pickle

Además del almacenamiento basado en la herramienta ROSbag, el sistem utiliza un mecanismo de almacenamiento persistente adicional para las trayectorias definidas y utilizadas en el sistema. Para ello, se ha creado un formato propio utilizando la librería **Pickle** de Python, almacenando las trayectorias en un mismo archivo con extensión **.data**.

El archivo **.data** contiene un diccionario que almacena todas las trayectorias definidas por el usuario, cada una identificada por un nombre único. La información de cada trayectoria se almacena en un formato estructurado, que incluye:

- **Nombre de la Trayectoria:** Identificador único para cada trayectoria, que por defecto es la fecha y hora de creación. Luego se puede modificar por el usuario, controlando que no haya duplicados.
- **Puntos de la Trayectoria:** Lista de puntos geográficos (latitud y longitud) que componen la trayectoria.
- **Representación de la trayectoria en el mapa:** Imagen almacenada en binario que muestra en un mapa de Google Maps la trayectoria definida por el usuario. Esta imagen se genera automáticamente al crear la trayectoria y se almacena junto con los puntos geográficos.

Esto permite una recuperación sencilla y rápida de trayectorias predefinidas, así como su reutilización en diferentes contextos experimentales o simulaciones futuras. La elección de Pickle se debe a su eficiencia y simplicidad, permitiendo una integración inmediata con el entorno Python utilizado en el sistema.

Ventajas del Modelo de Datos Seleccionado

La combinación de ROSbag para almacenamiento temporal y Pickle para almacenamiento persistente ofrece ventajas significativas, tales como:

- **Alta fiabilidad y precisión:** La precisión temporal y secuencial de ROSbag asegura una reproducción exacta de experimentos anteriores.
- **Reutilización efectiva:** La capacidad de almacenar y recuperar trayectorias rápidamente mediante archivos Pickle incrementa la eficiencia operativa y facilita experimentos repetibles.

3.3.5. Diseño de la interfaz gráfica

Conceptos Generales y Objetivos del Diseño

La interfaz gráfica del sistema ha sido diseñada con el objetivo de ofrecer una experiencia intuitiva y eficiente para el usuario final. Se buscó minimizar la curva de aprendizaje y garantizar que todas las funcionalidades clave del sistema son accesibles y comprensibles. Para ello, se han seguido principios de diseño centrado en el usuario, priorizando la usabilidad y la claridad visual.

Principios de Diseño y Usabilidad

Para asegurar una buena usabilidad, se han seguido principios fundamentales de diseño de interfaces:

- **Claridad y Simplicidad:** La información está presentada de manera clara y organizada, asegurando que el usuario pueda identificar rápidamente las acciones que puede realizar.
- **Consistencia:** Todos los elementos visuales y de interacción con el usuario mantienen una coherencia a lo largo de la interfaz, facilitando la navegación y el uso del sistema. Se han utilizado iconos y colores estandarizados para representar acciones y estados.

- **Feedback Visual Inmediato:** Cada acción del usuario genera una respuesta inmediata y evidente en la interfaz, mejorando la sensación de control sobre el sistema.
- **Accesibilidad:** Se han considerado aspectos de accesibilidad para garantizar que la interfaz sea usable por diferentes tipos de usuarios, incluyendo la navegación mediante teclado.

Descripción Detallada del Mockup y sus elementos

A continuación se presenta el *boceto* diseñado inicialmente para la aplicación, explicando en detalle cada uno de los elementos y secciones que lo componen.

■ Página de Inicio

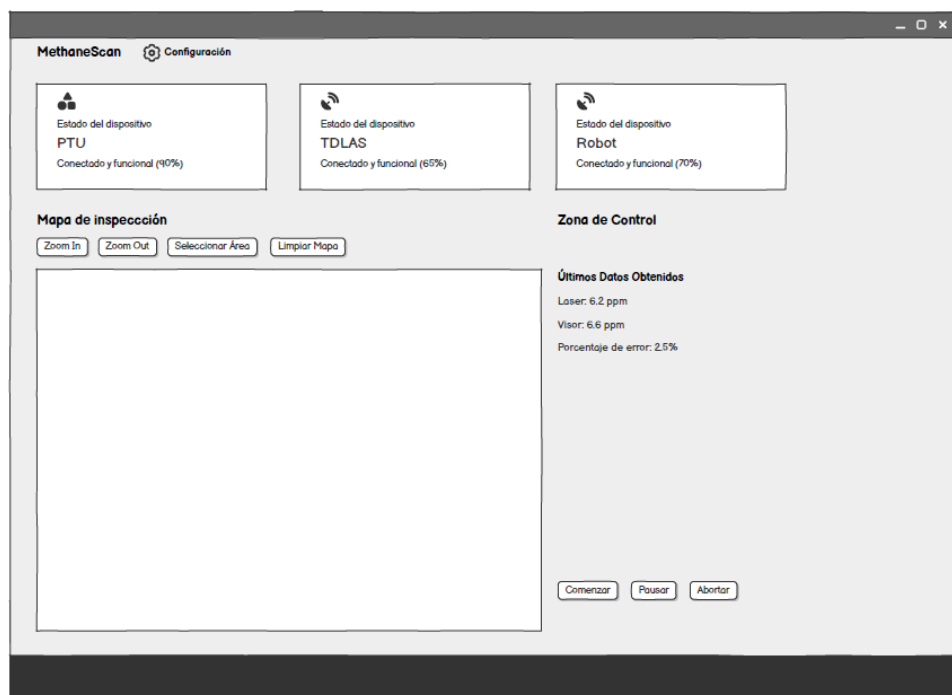


Figura 8: *Boceto* de la pantalla de inicio de la aplicación.

Este es el punto central de la aplicación. Está estructurada en secciones específicas para facilitar la rápida visualización del estado del sistema. En la parte superior se encuentran tarjetas informativas sobre el estado de los diferentes dispositivos actuales, desde las cuales podemos acceder a la configuración de cada uno de ellos. Debajo de estas se

encuentra la sección del mapa de inspección, permitiendo al usuario interactuar directamente con la visualización geográfica del área. En la parte derecha se encuentra la zona de control, incluyendo botones para controlar el experimento junto a una sección para mostrar los datos recibidos en tiempo real.

■ Configuración de los Dispositivos

The mockup shows a mobile application interface for 'MethaneScan' with a 'Configuración' (Configuration) tab selected. The main heading is 'Configuración de la PTU'. Below this, the 'Estado de la PTU' (PTU Status) section displays a battery icon at 70%, a 'Posición:' label with an empty text input field, and a shield icon indicating 'Estado: Operativo'. The 'Acciones' (Actions) section at the bottom contains two buttons: 'Aplicar Ahora' (Apply Now) and 'Descartar y volver' (Discard and return).

Figura 9: Boceto de la pantalla de configuración de la PTU.

The mockup shows a mobile application interface for 'MethaneScan' with a 'Configuración' (Configuration) tab selected. The main heading is 'Configuración del Robot'. Below this, the 'Estado del Robot' (Robot Status) section displays a battery icon at 70%, a 'Posición:' label with an empty text input field, a 'Velocidad:' label with an empty text input field, and a shield icon indicating 'Estado: Operativo'. The 'Acciones' (Actions) section at the bottom contains two buttons: 'Aplicar Ahora' (Apply Now) and 'Descartar y volver' (Discard and return).

Figura 10: Boceto de la pantalla de configuración del robot.

Estas pantallas permiten al usuario configurar los distintos parámetros disponibles para cada uno de los dispositivos individuales, presentadas mediante formularios simples y visualizaciones en tiempo real del estado de los dispositivos.

■ Simulación de Experimentos

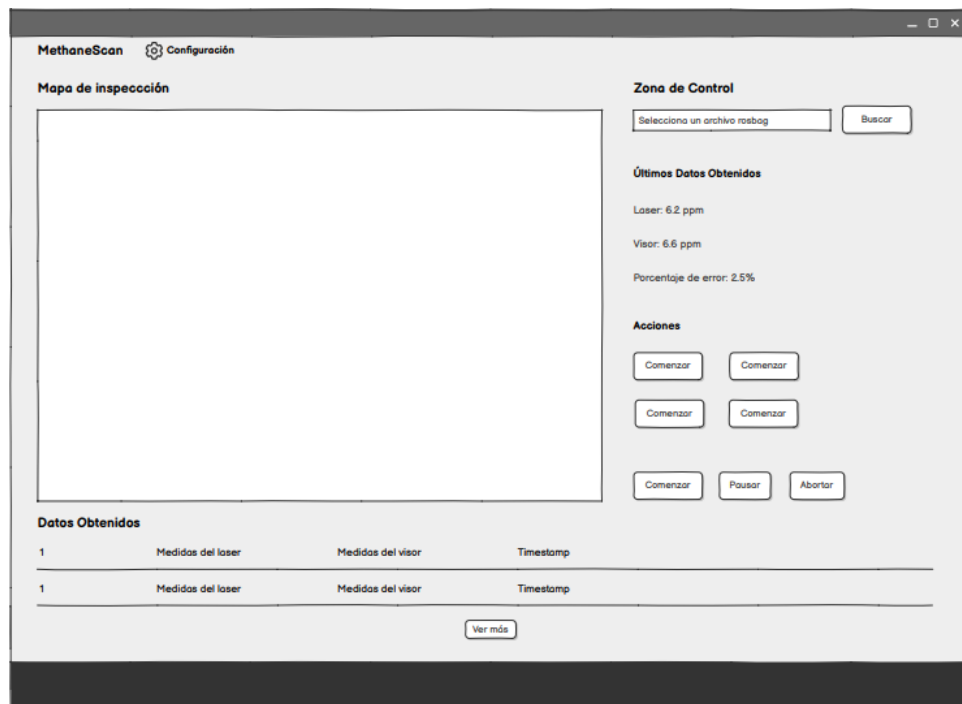


Figura 11: *Boceto* de la pantalla de simulación de experimentos.

Esta vista permite al usuario simular los experimentos registrados en el archivo rosbag, proporcionando una interfaz para la reproducción de los datos y la visualización de las mediciones en tiempo real. Incluye controles para pausar, avanzar y abortar en la simulación, así como opciones para ajustar la velocidad de reproducción. Además, se incluye una pequeña tabla de datos que va mostrandos la información de la medición de metano en tiempo real, así como la posición del robot.

3.3.6. Modelos UML del Sistema

Los modelos UML (Unified Modeling Language) permiten representar visualmente el comportamiento del sistema desde distintas perspectivas, facilitando la comprensión del flujo funcional y la interacción entre componentes. Para el sistema desarrollado, se han creado dos tipos de diagramas UML: el diagrama de casos de uso, el de actividades y el diagrama de secuencia.

Diagrama de Casos de Uso

El diagrama de casos de uso representa las interacciones entre los actores del sistema y las funcionalidades que ofrece. En este caso, se han identificado dos actores principales: el usuario

y el sistema. El diagrama muestra cómo el usuario interactúa con la interfaz gráfica para controlar los dispositivos, simular experimentos, gestionar trayectorias e iniciar el experimento.

3.3.7. Conclusión del Diseño del Sistema

El diseño del sistema propuesto presente una arquitectura coherente y alineada con los principios de ingeniería de software orientada a sistemas ciberfísicos y robóticos. A través del uso de ROS2 como infraestructura base, complementado con herramientas como PyQt5 y MQTT, se ha conseguido integrar un entorno robusto que satisface los requisitos planteados desde el inicio del proyecto.

La modularidad ha sido uno de los pilares fundamentales del diseño. Cada componente del sistema ha sido desarrollado como un módulo independiente con responsabilidades claramente definidas, lo que facilita en gran medida su mantenimiento y escalabilidad. Esto permite que en el futuro se puedan añadir nuevas funcionalidades complejas sin alterar el funcionamiento de los módulos existentes.

En términos de escalabilidad, el uso de ROS2 y la abstracción de la lógica en controladores nos proporciona una base sólida en la que podemos trabajar y extender de manera rápida y eficiente el sistema. De esta forma, se pueden añadir nuevos dispositivos, algoritmos o funcionalidades externas, incluyendo la conexión con plataformas IoT o servicios en la nube.

Para resumir, el diseño adoptado es coherente con los objetivos del proyecto, facilita la integración de nuevas tecnologías, permite un desarrollo evolutivo y responde adecuadamente a los desafíos técnicos propios de un sistema de medición remota y control distribuido.

4

Desarrollo e Integración Tecnológica

4.1. Entorno de Desarrollo y Estructura del Proyecto

El desarrollo del sistema se ha realizado sobre una arquitectura ROS2 utilizando Python como lenguaje de programación principal y el *framework* PyQt5 para la construcción de toda la interfaz gráfica. Todo el entorno ha sido desplegado en un sistema operativo Ubuntu 22.04 LTS, el cual es compatible con ROS2 Humble. Se ha elegido esta versión debido a su estabilidad y compatibilidad con herramientas modernas como ROSbag y rqt.

Entorno de trabajo y herramientas utilizadas En este apartado, comentaremos más a fondo las herramientas y entornos mencionados anteriormente.

- **ROS2 Humble:** Es el framework principal para la comunicación y gestión de nodos, proporcionando una arquitectura distribuida basada en tópicos, servicios y acciones. Es utilizado ya que, al utilizar elementos robóticos en el experimento, este entorno es el ideal para dichos sistemas.
- **Python 3.10:** Lenguaje principal de programación por su compatibilidad con ROS2 y su amplia biblioteca para manipulación de datos, interfaces gráficas y automatización. Además de su sintaxis clara y legible, Python proporciona una gran cantidad de librerías especializadas, lo que agiliza considerablemente el desarrollo de sistemas complejos como este.
- **PyQt5:** Es considerada una de las mejores librerías de interfaces gráficas para Python.

Es empleada para un diseño de vistas ricas y personalizables. Gracias al uso de QSS (Qt Style Sheets) es posible dicha personalización, ya que nos permite aplicar estilos visuales avanzados y altamente personalizados, de forma similar a las hojas de estilo de CSS en aplicaciones Web. Además, esta librería permite una integración directa con señales y slots de Qt, utilizadas para mantener la comunicación fluida con los nodos ROS.

- **Visual Studio Code:** Editor de código ligero y altamente configurable, utilizado con extensiones específicas para el desarrollo de Python, gestión de entornos virtuales y depuración de nodos ROS.
- **QWebEngine + HTML embebido:** Tecnología integrada en PyQt5 que permite renderizar contenido HTML interactivo dentro de una aplicación de escritorio. En este sistema, se utiliza para mostrar los mapas de Google Maps cargando archivos HTML con su correspondiente lógica en JavaScript. Esto permite utilizar la API de Google Maps dentro de aplicaciones, sin depender de navegadores externos como Chrome.
- **Git y Github:** Sistema de control de versiones distribuido y plataforma de repositorios para gestionar el código fuente, documentación y control de cambios.
- **Pickle / JSON:** Librerías para serializar los datos de trayectorias y configuraciones del sistema. JSON es utilizado para la interoperabilidad, mientras que Pickle se emplea para el almacenamiento estructurado local.
- **Balsamiq Wireframes:** Herramienta de prototipado rápido para la creación de maquetas visuales de la interfaz gráfica. Se utiliza para validar y ajustar el diseño antes de su implementación final.
- **Herramientas de depuración y análisis de ROS2:** Herramientas como `rqt`, `ros2 topic echo` son propias del ecosistema ROS y se utilizan para la monitorización de tópicos en tiempo real, publicación de mensajes, etc. Además, se han utilizado herramientas de grabación y reproducción de datos experimentales como `ros2 bag`.

Organización del proyecto y estructura modular

El sistema está estructurado en dos paquetes ROS2 principales:

- **methane_scan**: Este paquete incluye el nodo principal ROS2, la lógica de control y todas las vistas gráficas. Por simplicidad, explicaré la estructura omitiendo aquellos directorios comunes en cualquier paquete ROS2 (resources, test, etc).
 - **methane_scan**: Contiene el código fuente del paquete. Se explicará más adelante.
 - **launch/**: Contiene los archivos de lanzamiento para iniciar el nodo principal y los nodos auxiliares, además de los archivos de configuración de los parámetros del sistema.
- **mqtt_bridge**: Este paquete es un puente entre el sistema MQTT y ROS2. Se encarga de recibir los mensajes de los sensores y publicarlos en los tópicos correspondientes de ROS2.
 - **src**: Contiene el archivo principal del nodo, escrito en C++. Se encarga de actuar como puente entre el sistema MQTT y ROS2, publicando los mensajes recibidos en los tópicos correspondientes.

Una vez mostrado la estructura del proyecto de manera general, a continuación se explicará la estructura del paquete **methane_scan** en profundidad. Este paquete es el núcleo del sistema y contiene toda la lógica de control y la interfaz gráfica. La estructura de este paquete es la siguiente:

- **/controllers**: Incluye la lógica de control del sistema, que se encarga de gestionar la comunicación entre los distintos nodos y la interfaz gráfica. Además, contiene la lógica dedicada al control del robot, la PTU y el sensor de tecnología TDLAS.
- **/resources**: Contiene todos los archivos correspondientes a las imágenes e iconos utilizados en la interfaz gráfica.
- **/views**: Contiene todos los archivos de la interfaz gráfica, incluyendo los archivos QSS (Qt Style Sheets) y los archivos HTML para la visualización de mapas. Además, contiene el archivo principal de la interfaz gráfica y el archivo de la pantalla de arranque. Este directorio contiene los siguientes subdirectorios:
 - **/components**: Contiene componentes individuales de la interfaz gráfica, como puede ser la vista del mapa, las cartas de los dispositivos, los *toasts* de información, etc.

Cada componente está diseñado para ser reutilizable y modular, facilitando su integración en diferentes partes de la aplicación.

- `/pages`: Contiene las distintas pantallas de la aplicación, como la pantalla de inicio, la pantalla de configuración y la pantalla de simulación. Cada pantalla está diseñada para ser independiente y modular, facilitando igualmente su integración en la aplicación.
- `/web`: Contiene los archivos HTML y JavaScript necesarios para la visualización de mapas y la interacción con la API de Google Maps. Estos archivos son cargados en la aplicación mediante el componente QWebEngine, permitiendo una integración fluida con la interfaz gráfica.

Además, dentro de este directorio se encuentran el archivo principal de la interfaz gráfica (`main_window.py`) y el archivo de la pantalla de arranque (`splash_screen.py`). Estos archivos son los encargados de inicializar la aplicación y mostrar la interfaz gráfica al usuario. Otra parte que se encuentra aquí es el archivo de estilo general, `dark_style.qss`, que define el estilo visual de toda la aplicación. Este archivo es cargado al inicio de la aplicación y se aplica a todos los componentes de la interfaz gráfica, asegurando una apariencia coherente y atractiva.

Por último, en este paquete se encuentran los archivos ejecutables de los nodos, tanto el nodo principal (`methane_scan_node`) como el nodo auxiliar (`mqtt_ros_bridge_node`). El primero es el encargado de gestionar la interfaz gráfica y la lógica de control del sistema, asegurando una correcta adaptación al ecosistema ROS2. El segundo es el nodo que se encarga de recibir los mensajes provenientes del cliente MQTT y publicarlos en los tópicos correspondientes de ROS2, y viceversa, actuando como un puente entre ambos sistemas. Este nodo es fundamental para la integración del sistema, ya que permite la comunicación entre el cliente MQTT y el sistema ROS2, asegurando una correcta transmisión de datos y mensajes entre ambos sistemas.

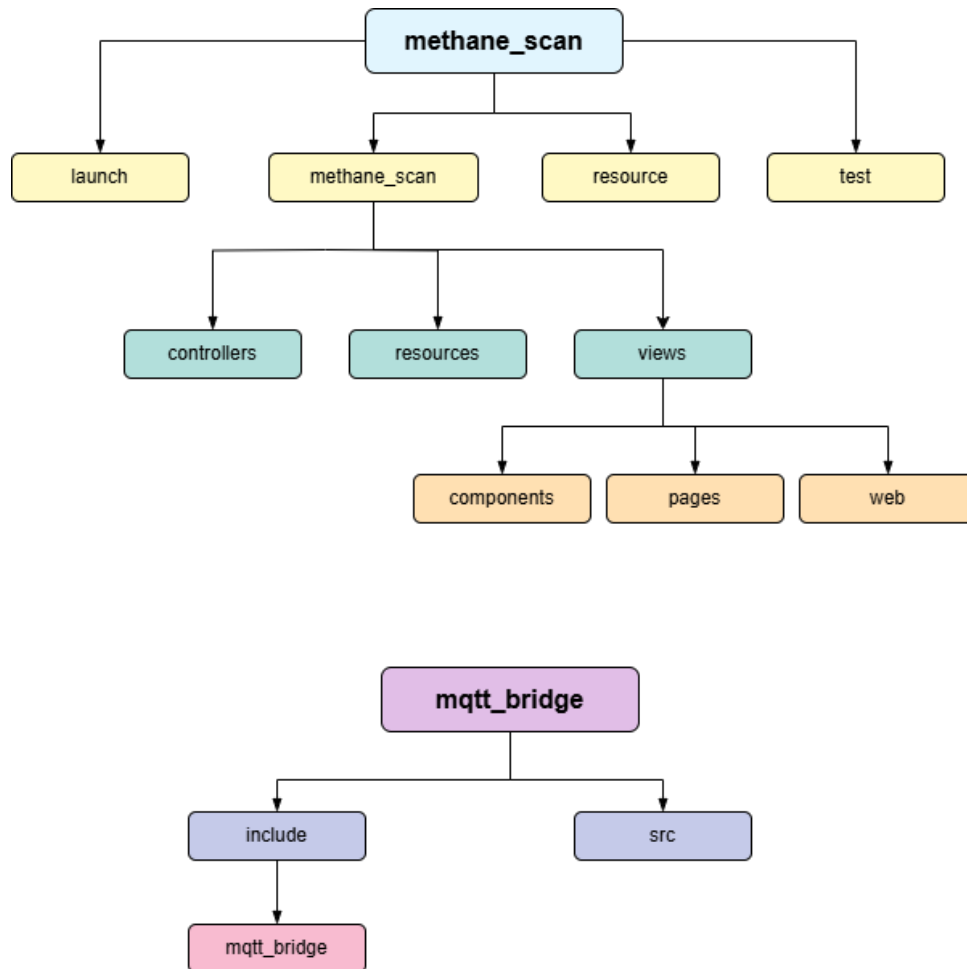


Figura 12: Estructura del proyecto

Esta estructura modular ha sido clave para mantener un código limpio y un desarrollo organizado, facilitando la navegación entre archivos, el despliegue controlado del sistema y permitiendo la integración de nuevas funcionalidades sin modificar la base existente.

4.2. Integración de la Interfaz Gráfica con ROS2

La integración entre la interfaz gráfica desarrollada en PyQt5 y los nodos ROS2 constituye uno de los elementos clave del sistema, ya que es lo que permite una interacción fluida y segura entre los datos sensoriales, los controles operativos y la experiencia del usuario. Esta integración se ha logrado mediante el uso de señales y slots, una característica fundamental de Qt que permite la comunicación entre diferentes componentes de la aplicación.

Comunicación entre señales Qt y callbacks ROS2

En Qt, la comunicación entre componentes se basa en el mecanismo de señales y slots. Una **señal** es un mensaje emitido por un objeto cuando ocurre un evento específico, mientras que un **slot** es una función que se ejecuta en respuesta a una señal. Este modelo permite una arquitectura desacoplada, donde los componentes pueden comunicarse sin necesidad de conocer los detalles internos de cada uno.

En PyQt5, las señales se definen mediante la clase `pyqtSignal`, que permite crear señales personalizadas y emitir cualquier tipo de dato. Cuando una señal se conecta a un slot (por ejemplo, una función del controlador o de la vista), al emitirse, el slot se ejecuta automáticamente con los datos proporcionados. Esta característica es especialmente útil para enlazar los eventos asincrónicos, como la llegada de mensajes ROS2, los cuales conllevan en su mayoría una actualización gráfica o un cambio en el estado de la aplicación.

En el sistema desarrollado, las señales se utilizan principalmente para trasladar de forma segura la información desde los hilos de ROS2 al hilo principal de la interfaz gráfica, además de la conexión entre los distintos componentes de la aplicación. Con esto se garantiza que todas las modificaciones en los widgets de PyQt se realicen de forma *thread-safe*, evitando problemas de concurrencia y asegurando una experiencia de usuario fluida y sin bloqueos.

5

Conclusions and Futures Lines of Research

5.1. Conclusions

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

5.2. Future lines of Research

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna

dictum turpis accumsan semper.

Conclusiones y Líneas Futuras

6.1. Conclusiones

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

6.2. Líneas Futuras

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Referencias

- [1] *Diagrama funcional de la red de monitorización*. https://www.researchgate.net/figure/Figura-1-Diagrama-funcional-de-la-red-de-monitorizacion_fig1_342618983. Consultado: 17 de abril de 2025. 2020.
- [2] H. He et al. «In-situ testing of methane emissions from landfills using laser absorption spectroscopy». En: *Applied Sciences* 11.5 (2021), pág. 2177. DOI: [10.3390/app11052117](https://doi.org/10.3390/app11052117). URL: <https://doi.org/10.3390/app11052117>.
- [3] International Energy Agency. *Sources of methane emissions, 2023*. <https://www.iea.org/data-and-statistics/charts/sources-of-methane-emissions-2023-2>. Licence: CC BY 4.0. 2024.
- [4] Steven Macenski et al. «Robot Operating System 2: Design, architecture, and uses in the wild». En: *Science Robotics* 7.66 (2022), eabm6074. DOI: [10.1126/scirobotics.abm6074](https://doi.org/10.1126/scirobotics.abm6074). URL: <https://www.science.org/doi/10.1126/scirobotics.abm6074>.
- [5] Kathleen A. Mar et al. «Beyond CO₂ equivalence: The impacts of methane on climate, ecosystems, and health». En: *Environmental Science & Policy* 134 (2022), págs. 127-136. ISSN: 1462-9011. DOI: <https://doi.org/10.1016/j.envsci.2022.03.027>. URL: <https://www.sciencedirect.com/science/article/pii/S1462901122001204>.
- [6] V. Masson-Delmotte et al. *Climate Change 2021: The Physical Science Basis. Contribution of Working Group I to the Sixth Assessment Report of the Intergovernmental Panel on Climate Change*. Cambridge University Press, 2021. DOI: [10.1017/9781009157896](https://doi.org/10.1017/9781009157896). URL: <https://www.ipcc.ch/report/ar6/wg1/>.
- [7] ResearchGate. *Gas Sensor Using a Robust Approach under Time Multiplexing Scheme with a Twin Laser Chip for Absorption and Reference - Scientific Figure*. https://www.researchgate.net/figure/Schematic-and-photo-of-the-TDLAS-gas-sensor-prototype_fig1_228875523. Accessed: 2025-04-17.
- [8] Panagiotis Siozos, Giannis Psyllakis y Michalis Velegrakis. «Remote Operation of an Open-Path, Laser-Based Instrument for Atmospheric CO₂ and CH₄ Monitoring». En:

Photonics 10.4 (2023). ISSN: 2304-6732. DOI: [10.3390/photonics10040386](https://doi.org/10.3390/photonics10040386). URL: <https://www.mdpi.com/2304-6732/10/4/386>.

- [9] U.S. Environmental Protection Agency. *Inventory of U.S. Greenhouse Gas Emissions and Sinks: 1990-2020*. Inf. téc. EPA 430-R-22-003. U.S. Environmental Protection Agency, 2022. URL: <https://www.epa.gov/ghgemissions/inventory-us-greenhouse-gas-emissions-and-sinks-1990-2020>.

Apéndice A

Manual de Instalación



UNIVERSIDAD
DE MÁLAGA

| **uma.es**

E.T.S. DE INGENIERÍA INFORMÁTICA

E.T.S de Ingeniería Informática
Bulevar Louis Pasteur, 35
Campus de Teatinos
29071 Málaga